

MVP BUILD SPECIFICATION

Phase 0

Telegram-Only Expense Tracker

Transport Official Telegram Bot API

Extraction PaddleOCR — local, no paid AI API

Database MongoDB

Status Specification — not yet implemented

Supersedes Phase 0 — WhatsApp-Only Expense Tracker (on hold)

Goal of this phase — unchanged:

Do people who send us a receipt keep sending receipts?

Contents

1. Scope
2. Architecture — Telegram Bot API
3. Conversational Layer — Feeling Like An Assistant, Not A Command Bot
4. Data Model — MongoDB
5. Receipt Data Extraction — Local, Free, Self-Hosted
6. Categorization
7. Deduplication — Closing The Loopholes
8. Handling Junk / Non-Receipt Input
9. Message Types The Bot Needs To Produce
10. What "Done" Looks Like For Phase 0
11. Explicitly Deferred To Phase 1+ (Do Not Build Now)
12. Transport Migration — Planned, Not Built

Status note: The WhatsApp version of Phase 0 is **on hold**, not cancelled. It was shelved because `whatsapp-web.js` required a real WhatsApp account, a persistent Chromium/Puppeteer session, session persistence across restarts, a continuously running linked session, and carried a real risk of the number being flagged — too much operational overhead for a first validation phase. Telegram gives us an official, supported bot API with none of that. See Section 12 for how we get back to WhatsApp later.

Goal of this phase — unchanged: answer one question — **do people who send us a receipt keep sending receipts?**

No app. No dashboard. No account creation flow beyond identifying a Telegram account as a user. Everything lives inside a Telegram chat.

If this phase doesn't get repeat usage, nothing downstream (dashboard, app, multi-bank polish) matters yet. Build only what's needed to test that.

1. Scope

In:

- Receive receipt image/PDF/document in a Telegram chat
- Extract: amount, merchant, date, transaction type, reference number
- Categorize automatically
- Ask for category when uncertain
- Deduplicate
- Reply with a confirmation
- Understand a spending summary request in the user's own words, not just a fixed "summary" keyword (see Section 3)
- Understand a request to undo/correct the last entry in the user's own words, not just a fixed "undo" keyword (see Section 3) — you will need this, someone will send a test/junk image

Out (explicitly, for this phase — not just "later," but not built):

- Mobile/web app, dashboard UI
 - Phone verification / signup flow (a user is just "whatever Telegram account messaged the bot" — no formal account creation)
 - Budgets, goals, predictions, financial advice
 - Bank connections, open banking, Mono/Okra, email scraping
 - Multi-user households, shared accounts, business use
 - Editing amount/merchant/date (only category is user-correctable — anything else wrong means the parse failed and should be logged for you to review, not patched by the user)
-

2. Architecture — Telegram Bot API

Why Telegram for Phase 0

The Telegram Bot API is an **official, documented, supported** API. That removes the entire operational category of problems that stalled the WhatsApp version:

- **No browser automation.** The backend makes ordinary HTTPS calls to Telegram's API. There is no headless Chromium, no Puppeteer, no page scraping.
- **No linked-session maintenance.** Authentication is a bot token issued by [@BotFather](#), stored in the `TELEGRAM_BOT_TOKEN` environment variable. No QR scanning, no session files to keep alive, no re-linking after a restart.
- **No Terms-of-Service risk.** Bots are a first-class, intended feature of the platform. There is no risk of the account being flagged for automation, because automation is the point.
- **No dedicated phone number or spare SIM.** A bot is created from any Telegram account and then runs independently of it.
- **Deployment is flexible.** Long polling works on a plain always-on process; webhooks work on a normal HTTPS request/response host, including serverless. Either is acceptable — **start with long polling**, because it needs no public URL, no TLS certificate, and no tunnelling during local development. Webhooks are a later optimisation, not a Phase 0 requirement.

Layering — the thing to get right

The single most important structural rule in this spec:

The transport layer is a thin adapter. The expense-tracking core knows nothing about Telegram.

We already changed transport once. We will likely change it again (see Section 12). Anything that survives a transport change must not import a Telegram library.

Transport-specific (thin, replaceable):

- Receiving updates from Telegram
- Downloading file bytes via `getFile` + the file-download endpoint
- Sending replies
- Mapping a Telegram user to a `telegram_user_id`

Transport-independent (the actual product):

- Receipt extraction
- Receipt validation
- Categorization
- Deduplication
- Transaction storage
- Summary generation
- Undo/correction
- Conversational intent handling
- Junk/non-receipt handling

- Logging and debugging

In practice: the adapter converts an incoming Telegram update into a neutral internal object — roughly `{ user_id, kind: 'media' | 'text', text?, file_bytes?, mime_type?, file_name? }` — hands it to the core, gets back a neutral reply object, and sends it. Swapping Telegram for something else should mean writing a new adapter, not touching the pipeline.

End-to-end flow

```

User
↓
Telegram
↓
Telegram Bot (official Bot API – long polling or webhook)
↓
Expense Tracker Backend ← transport adapter boundary is here
↓
Receipt Processing (local, see Section 5)
↓
Database (MongoDB, see Section 4)
↓
Telegram Reply

```

Message router

The router uses **intent classification for plain text, not exact keyword matching** — see Section 3 for why and how.

```

Telegram Bot
|
| incoming update
▼
Message Router
├─ image / PDF / document      → Receipt pipeline
├─ text = pending category reply → Category correction handler
├─ text intent: request_summary → Summary handler
├─ text intent: request_undo    → Undo handler
├─ text intent: small_talk      → Conversational reply
└─ text intent: unclear         → Conversational fallback (Section 3)
▼
Receipt pipeline (transport-independent):
1. Receive file bytes + mime type from the adapter
2. Store the raw payload immediately, before any processing
3. Fast-path dedup on raw file hash (Section 7)
4. Local extraction – PaddleOCR (Section 5)
5. Field extraction + validation, amount especially (Section 5)
6. Is-this-actually-a-receipt decision (Section 8)
7. Content-based dedup check (Section 7)
8. Categorization (Section 6)
9. Save transaction (status: confirmed | needs_category)
10. Return confirmation text → adapter sends it

```

Telegram media handling, concretely

Telegram delivers files three ways, and the bot must handle all of them, because testers will use all of them:

- **photo** — an image sent as a photo. Telegram provides several sizes; **always take the largest**, since smaller ones are recompressed and will hurt OCR accuracy.
- **document** — an image or PDF sent as a file (Telegram preserves this without recompression, so it is the *better* input when available). Check `mime_type` to distinguish `image/*` from `application/pdf`.
- Anything else with a file attached that isn't an image or PDF → treat as junk per Section 8, don't attempt extraction.

Downloading is two steps: call `getFile` with the `file_id` to get a `file_path`, then fetch the bytes from the file-download endpoint. The adapter does this and hands raw bytes to the core. **The core never sees a `file_id`.**

You need, minimum

- A Telegram bot created via BotFather, token in `TELEGRAM_BOT_TOKEN`
- One backend process (long polling — no public URL needed)
- **MongoDB**, connection string in `MONGODB_URI` — one `users` collection, one `transactions` collection (Section 4)
- A local Python environment running PaddleOCR, callable from the backend (Section 5)
- **No paid AI API. No API key for extraction or categorization.**
- No queue/worker infrastructure at test-group scale — synchronous processing is fine. One caveat: local OCR takes seconds, not milliseconds, so send Telegram a "typing" chat action while processing so the user isn't staring at silence.

3. Conversational Layer — Feeling Like An Assistant, Not A Command Bot

The goal is for this to read like messaging someone who's quietly keeping track of your spending, not like typing commands into a bot. Telegram makes slash-commands very easy and very tempting. **Resist that.** `/summary` and `/undo` may exist as undocumented conveniences, but they are not the interface — natural language is.

The boundary to hold: warmth and natural-language understanding change how the bot listens and talks. They do **not** add opinions, recommendations, warnings, or predictions about the user's spending. The bot still only does what's listed in Section 1 — log, categorize, summarize, undo — it just does it conversationally instead of via commands. If "give proactive spending advice" becomes something you want, that's a deliberate Phase 2+ decision, not something that should sneak in through tone work.

Understand intent, not exact commands

Every plain-text message that isn't a pending category reply goes through a **lightweight local intent classifier**. Classify into a small fixed set: `request_summary`, `request_undo`, `small_talk` (greetings, "you there?"), and `unclear`.

Because there is no cloud LLM in this architecture, intent classification is **local**. Approach, in order of preference:

1. **Normalized phrase and pattern matching over a deliberately broad surface.** Lowercase, strip punctuation, then match against generous keyword/phrase sets with fuzzy tolerance for typos — not against three literal strings. `request_summary` should catch *spent, spending, how much, total, this month, summary, breakdown, what am I spending on*. `request_undo` should catch *remove, delete, undo, take that out, that was wrong, wasn't right, cancel that, last one*. This is more than adequate for the phrasings a small test group actually uses, and it is deterministic, instant, free, and debuggable.
2. **Optional: a small local model** (e.g. a quantized instruct model via llama.cpp) if (1) proves insufficient once you see real messages. **Do not build this up front.** Log every `unclear` message during testing — that log tells you whether you need it. Most likely you won't.

Anything that doesn't map cleanly falls into `unclear` — **don't force a guess** (same "ask, don't guess" principle as extraction and categorization, applied here too).

- Example phrasings → `request_summary`: "how much have I spent this month", "what's my spending looking like", "what am I spending most on", "summary", "how much did I use on food".
- Example phrasings → `request_undo`: "remove that", "that one was wrong", "delete the last one", "undo the last transaction".

Because this is interpretation rather than exact match, **add a lightweight confirm-before-acting step when undo intent confidence is low** — you don't want a misread message accidentally deleting a real transaction.

Tone and response design

- Replies should read like a person, not a template: vary phrasing, echo back the merchant/amount naturally, keep it brief — personal doesn't mean chatty or long.
- Confident log example: "Got it — ₦12,500 to Jumia, tagged as Shopping ✅"
- Needs category example: "Saw a ₦12,500 charge to Jumia but wasn't sure where to file it — what would you call this one?"
- Summary example: "Here's where things stand this month — Food ₦35,000, Shopping ₦24,500, Transport ₦18,000. Total: ₦77,500."
- Undo example: "Done — took that ₦12,500 Jumia entry back out."
- Small talk/greeting: reply naturally and briefly, then let the user lead. Don't force every reply back to "send me a receipt" — that's what makes something feel like a bot.
- Keep several natural phrasing variants per response type rather than one fixed string, so it doesn't feel copy-pasted across messages. **The exact amount, merchant, and category must always be accurate — vary how it says it, never what it says.**
- Note on `/start`: Telegram sends `/start` when a user first opens the bot. Answer it in the same warm tone (a one-line "send me a receipt screenshot and I'll keep track" is plenty), not with a command menu.

The honest fallback for unclear intent

If intent classification comes back `unclear`, don't force it into the nearest category — reply conversationally asking what they meant, in the same tone as everything else, not a rigid "invalid command" error.

Example: "Not sure I caught that — did you want your spending summary, or is something wrong with a transaction I logged?"

The line we're not crossing

- **No unsolicited spending commentary, tips, warnings, or judgments** — it should never volunteer "you're spending a lot on shopping this month" unprompted. That's financial coaching, and it stays out of scope.
- No predictions, no budget suggestions, no "you should" language — even where it would sound natural for a warm assistant to say it.
- The warmth lives in phrasing and listening, not in opinions about the user's money.

4. Data Model — MongoDB

MongoDB is the database for Phase 0. Connection string in `MONGODB_URI`.

The reasoning: this is a persistent backend, and the receipt pipeline produces raw extraction output whose shape will change as extraction improves. A document store absorbs that without migrations.

`raw_extraction_json` in particular is a natural document field rather than a serialized blob in a text column.

Keep all database access behind one module/service. The rest of the application calls things like `saveTransaction()`, `findRecentTransaction()`, `getMonthlySummary()` — it does not import a MongoDB driver, does not build queries inline, and does not pass around driver-specific objects. Same principle as the transport adapter: the database is a replaceable detail, not an architectural assumption.

Collection: `users`

Field	Notes
<code>telegram_user_id</code>	Primary user identifier (Telegram's numeric user ID). Unique.
<code>created_at</code>	First message seen

A user record is created on first contact. There is no signup, no verification, no auth.

(Implementation note: the "does this user have a pending category question" state from Section 6 can live either as a field here or as a lookup for the user's most recent `needs_category` transaction. Either is fine — don't add a collection for it.)

Collection: `transactions`

Field	Notes
<code>_id</code>	MongoDB document ID
<code>telegram_user_id</code>	Owner
<code>amount_kobo</code>	Integer kobo, never a float naira value — avoids rounding bugs
<code>merchant</code>	Raw extracted string

Field	Notes
<code>transaction_date</code>	Date
<code>transaction_type</code>	payment / transfer / airtime / etc — whatever the receipt says
<code>reference_number</code>	Nullable — not all receipts have one
<code>category</code>	Nullable until confirmed
<code>status</code>	<code>confirmed</code> <code>needs_category</code> <code>rejected</code> (Section 8)
<code>source_receipt_hash</code>	Raw file hash (Section 7)
<code>raw_extraction_json</code>	Always store the full extraction output — you will need this for debugging parse failures
<code>created_at</code>	

Indexes

- `telegram_user_id` — every query filters on it
- `reference_number` — dedup lookups
- `source_receipt_hash` — fast first-pass dedup
- `transaction_date` — monthly summaries
- Compound `{ telegram_user_id: 1, transaction_date: -1 }` is worth adding, since summary and undo both read "this user, most recent first"

Operations the database layer must support

- Create a transaction
- Update a transaction's category
- Duplicate detection lookups (by file hash, by reference number, by amount + merchant + date within a user)
- Retrieve and undo the most recent valid transaction for a user
- Monthly spending summary grouped by category for a user
- Retrieve a user's transaction history

Do not add collections beyond `users` and `transactions`, an authentication system, or any additional database infrastructure. Rejected/failed extractions are logged per Section 8 — either as `status: rejected` transaction documents excluded from all summaries, or to a log file. Don't build a third collection for them unless the log file genuinely proves unworkable.

5. Receipt Data Extraction — Local, Free, Self-Hosted

The direction change

The previous version of this spec called for a cloud vision LLM (Google Gemini) to read receipts. **That is no longer the architecture.** Phase 0 now runs on a completely free/open-source/local stack, with no per-receipt API cost and no dependency on a paid provider.

No cloud LLM — Gemini, OpenAI, Claude, or otherwise — is a required part of this architecture.

Architecture

```

Receipt image / PDF
↓
PaddleOCR (local)
↓
Extracted text + layout / bounding boxes
↓
Nigerian receipt extraction + validation logic
↓
Structured transaction

```

PaddleOCR is the primary extraction foundation. It runs locally, is free, handles the input we actually get, and — critically — returns per-item confidence scores and bounding boxes, which is what makes the validation in this section possible.

Why this works for our input

Our receipts are overwhelmingly **screenshots of Nigerian banking apps** — GTBank, Kuda, OPay, PalmPay. That means digitally rendered text, high contrast, consistent fonts, no camera skew, no paper curl, no shadows. This is close to the ideal case for OCR, and quite different from photographing a crumpled paper till receipt.

There is also a real advantage over a vision LLM for this specific product. A vision model can return ₦150,000 for a ₦15,000 receipt while reporting high confidence — wrong *and* plausible, with nothing to catch it. OCR fails visibly instead: garbled characters, low confidence scores, malformed output. Given that a wrong amount is the one failure mode called out as unacceptable in this spec, **a failure mode you can detect beats a slightly lower raw error rate.**

Field extraction — label anchoring, not per-bank templates

Do not build `gtbank.ts`, `opay.ts`, `kuda.ts`. That is exactly the brittle template-per-bank system this spec is trying to avoid, and it breaks every time a bank redesigns its app.

Use **general extraction rules over OCR layout** instead. PaddleOCR returns text with bounding boxes, so anchor on labels rather than positions: find the text "Amount" / "Amount Paid" / "Total", then take the currency-formatted number nearest to it spatially. Every Nigerian banking app labels its fields — they just arrange them differently. Label-relative extraction survives a UI redesign; a line-index template does not.

Fields to recognize (all nullable):

- **amount** — the critical one

- **transaction_date**
- **merchant / recipient**
- **sender**, where available
- **transaction_type** (payment, transfer, airtime, etc.)
- **transaction / reference / session ID**, where available — Nigerian receipts use all three labels
- **currency**

Never invent a field that isn't visible. If a value can't be read confidently, return `null` and follow the "ask, don't guess" rule. This applies with equal force here as it did under the previous architecture.

Bank-specific tuning is allowed **later**, once the real test set shows actual failure patterns — not before.

PDFs

Check for an embedded text layer first (`pdfplumber`, `PyMuPDF`, or similar). Bank-emailed receipts often have real selectable text — extracting it is exact, instant, and free of OCR error entirely. Only fall back to rasterizing page 1 and running OCR when there's no usable text layer. Beyond that first check, **don't build a separate PDF extraction path** — it converges into the same pipeline.

Validation — amount is the field to be paranoid about

This is now an explicit pipeline step, not an afterthought. Validate mechanically, before anything is saved:

- Amount matches an expected currency shape (`₦/NGN` optional, digit groups, at most 2 decimals)
- **Digit-count sanity check** — an extracted amount with more than ~7 digits is far more likely an OCR error than a real ₦10m+ transaction. Don't auto-confirm it.
- OCR confidence on the amount region is above threshold
- Date parses and is not in the future
- Reference number is alphanumeric of plausible length

When amount validation fails or confidence is low, do not auto-confirm. Route it to human confirmation the same way a low-confidence category does — ask the user, or ask for a clearer image. **Start conservative (confirm more often) and loosen it as you build trust in accuracy.**

Optional fallback: PaddleOCR-VL

For receipts where ordinary OCR is genuinely insufficient — unusual layouts, poor-quality photos of a screen — **PaddleOCR-VL may be used as an optional local document-understanding fallback.** It stays local; it does not reintroduce a cloud dependency.

Treat it as a fallback, not the default path: it is slower and heavier. **Don't build it in step one.** Get baseline OCR working, run the test set, and see whether the failures actually justify it.

Keep extraction behind one interface

Same principle as transport and database: the pipeline calls one `extractReceipt(bytes, mimeType)` function. Whether that's PaddleOCR, PaddleOCR-VL, or something else later is an internal detail. Swapping it should be a contained change, not a rewrite.

(Practical note: PaddleOCR is Python; the backend is Node. Run OCR as a small local service the backend calls over localhost, or as a child process. Either is fine — it's an implementation detail behind the interface above. Don't add a message queue for it.)

The single highest-value thing you can do before testing with real users

Build a test set of 20–30 real receipt screenshots across GTBank, Kuda, OPay and PalmPay, and run extraction against all of them before opening this to testers. Check amount accuracy specifically — a wrong amount (₦15,000 read as ₦150,000) is the unacceptable failure. Don't skip this to move faster; it's the cheapest insurance you have, and under a local-OCR architecture it's also how you find out whether you need the VL fallback at all.

The four banks are the **initial** test set, not a hard-coded system boundary. The architecture must accommodate additional Nigerian banks and fintechs without structural change.

6. Categorization

Assign one of the fixed categories: **Food, Shopping, Transport, Data & Airtime, Bills, Family, Savings & Investing, Health, Entertainment, Others**.

Since there is no cloud LLM, categorization is **local and rule-based**, driven by the extracted merchant string plus transaction type:

- Maintain a merchant → category mapping with normalized matching (lowercase, strip punctuation and common suffixes like "LTD", "NIG", "POS", terminal IDs). Nigerian receipts are highly repetitive — a modest list covers most real spend (e.g. Bolt/Uber → Transport, MTN/Glo/Airtel/9mobile → Data & Airtime, Chicken Republic/Jumia Food → Food, DSTV/GOTV/IKEDC → Bills).
- Transaction type is a useful secondary signal — an airtime purchase is Data & Airtime regardless of merchant string.
- **Produce an explicit confidence value**, same pattern as extraction. A direct mapping hit is confident; a fuzzy partial match or an unknown merchant is not.
- **Learn from the user, per user**. When a user answers a category question, remember that merchant→category choice for that user so the same merchant is auto-categorized next time. This is the mechanism that makes rule-based categorization improve with use instead of staying static — and it costs nothing.

If confident → **status: confirmed**, category set.

If not confident → **status: needs_category**, category left null, and send the clarifying message. Store that this Telegram user has a pending category question so the next plain-text reply from them is interpreted as the category answer, not routed to intent classification.

Validate the user's reply against the fixed category list (case-insensitive / simple fuzzy match). If it doesn't match anything, re-ask once with the list spelled out; **don't loop forever** — after one failed re-ask, default to "Others" and tell them it can still be changed by contacting you directly (manual, out of band — no in-product edit flow needed yet).

(Telegram offers inline keyboard buttons, which would make category selection one tap. This is tempting and would work — but it pushes the interface toward "bot with buttons" and away from the conversational feel Section 3 is protecting. Accept a typed reply either way, whatever you decide about buttons.)

7. Deduplication — Closing The Loopholes

The easy case is the same receipt sent twice. Here's what a real dedup check needs to cover. **All of this is transport-independent** — it operates on extracted fields and file bytes, and knows nothing about Telegram.

Fast first pass — raw file hash

Hash the raw file bytes (`source_receipt_hash`). If the identical file was sent again, **short-circuit immediately, before running OCR at all**. Under a local-OCR architecture this saves seconds of processing per duplicate, which matters more than it did when the cost was an API call.

Primary check — content-based, not just file-based

Two different images of the same transaction (a photo and a screenshot, or the same screenshot re-compressed by a different upload path) will not hash identically. So dedup on the extracted transaction fingerprint:

```
fingerprint = hash(telegram_user_id + amount_kobo + normalized_merchant
                  + transaction_date + reference_number)
```

- If **reference_number** is present and matches an existing transaction for that user → duplicate. Reject without creating a second document; reply "already logged this one .
- If **reference_number** is missing (some receipts don't have one) → fall back to matching on amount + merchant + date within the same user. Flag as **likely** duplicate rather than certain — reply asking "This looks like the same ₦X to [merchant] from [date] you already logged — is this a new transaction?" rather than silently rejecting, since two genuinely separate purchases can share amount + merchant + date (e.g. two identical ₦500 transport payments the same day).

Edge cases to explicitly handle, not ignore

- Same amount, same merchant, **different day** → not a duplicate, log normally.
- Same amount, **different merchant**, same day → not a duplicate.
- Extraction slightly disagrees between two sends of the truly same receipt (e.g. reference number OCR'd differently) → this is exactly why the amount + merchant + date fallback matters, not just reference-number matching.

8. Handling Junk / Non-Receipt Input

Someone will send: a selfie, a random screenshot, a meme, a blank or corrupted image, an unrelated document, a receipt from a non-bank source, or a receipt in a currency/format the system can't parse. Telegram makes forwarding trivial, so expect more of this, not less.

The is-this-a-receipt decision is now made from **OCR output**, not from a model's self-reported flag. It's a check the pipeline runs: does the extracted text contain the markers of a transaction receipt — a currency amount, a date, and recognizable transaction vocabulary (*amount, transaction, successful, reference, transfer, recipient, session*)? A selfie yields almost no text; a random screenshot yields text with none of these markers.

- **Not a receipt** → reply along the lines of: "That doesn't look like a transaction receipt — send a screenshot of a bank or fintech payment confirmation, or just ask me how much you've spent." Do not create a normal transaction document. Optionally create one with **status: rejected** purely for your own debugging visibility — **it must never count in summaries**.
- **Looks like a receipt, but the amount couldn't be extracted or failed validation** → don't guess, and don't silently drop it either. Reply asking for a clearer image, and store nothing until it's resent.
- **Unsupported or corrupted file** (non-image, non-PDF attachment, unreadable bytes) → handle gracefully with the same friendly reply. Never crash the process on a bad upload.
- **Log every rejected/failed extraction** — the raw file plus the full raw OCR output — somewhere you can review. This is your best source of "what's still broken" during testing, more useful early on than the summary feature itself, and under a local-OCR architecture it's also your evidence base for whether the PaddleOCR-VL fallback is worth adding.

9. Message Types The Bot Needs To Produce

See Section 3 for tone and phrasing. This is the checklist of situations needing a reply. Keep the short-and-warm feel; **vary the exact wording rather than using these as fixed strings**.

- **Confident log:** "Got it — ₦12,500 to Jumia, tagged as Shopping 
- **Needs category:** "Saw a ₦12,500 charge to Jumia but wasn't sure where to file it — what would you call this one?"
- **Duplicate (certain):** brief, friendly "already got this one" — no new document created
- **Duplicate (likely, needs confirmation):** as in Section 7
- **Not a receipt:** as in Section 8
- **Unreadable receipt:** ask for a clearer image, store nothing
- **Undo:** "Done — took that ₦12,500 Jumia entry back out."
- **Undo, low confidence:** confirm before acting — "Just to check, you mean the ₦12,500 Jumia one?"
- **Unclear intent:** as in Section 3's fallback
- **First contact / /start:** warm one-liner, not a command menu

Summary reply example (from a natural-language ask, not a fixed keyword):

This month so far:

```

Food — ₦35,000
Shopping — ₦24,500
Transport — ₦18,000

Total: ₦77,500

```

Undo behaviour: identify the most recent valid transaction for that Telegram user (excluding already-undone and **rejected** ones) and remove it safely. Keep the low-confidence confirmation step from Section 3.

10. What "Done" Looks Like For Phase 0

A tester can, entirely inside Telegram:

3. Send a receipt image or PDF from one of the initial Nigerian banks/providers (GTBank, Kuda, OPay, PalmPay)
4. Get a correct amount/merchant/date extraction — *verified against your own test set, not just "seemed to work"*
5. Get auto-categorized, or get asked and reply with a category
6. Send the same receipt twice and have it recognized as a duplicate
7. Send junk and have it rejected without polluting their transaction data
8. Ask for their spending in their own words and get an accurate summary back
9. Say something wrong got logged, in their own words, and have the bot undo it
10. Keep doing this across multiple days without the bot breaking or losing their history

What you're measuring: not feature completeness — repeat usage.

Track, per tester:

- Receipts sent
- Active days
- Week-1 activity
- Week-2 activity
- Repeat receipt submissions

That retention curve is the actual output of this phase, more than the code.

11. Explicitly Deferred To Phase 1+ (Do Not Build Now)

- Dashboard / app
- Phone-number verification / signup flow
- Direct upload fallback
- Budgets, goals
- Any official bank integration

- Multi-bank confidence tuning beyond the four initial banks
 - Bank-specific extraction templates (only if the test set proves them necessary)
 - Editing anything other than category
 - Spending predictions, financial advice
 - Additional transports (Section 12)
 - Webhook deployment, queues, worker infrastructure
-

12. Transport Migration — Planned, Not Built

WhatsApp is on hold, not abandoned. If Phase 0 shows repeat usage and WhatsApp turns out to be where users actually are, the migration path is the **official WhatsApp Business Cloud API** (Meta direct, or a provider like Twilio / 360dialog / Gupshup) — not `whatsapp-web.js` and not browser automation.

The work that makes this cheap is being done now, in Section 2's layering: **if the expense-processing core never imports a Telegram library, the migration is a new adapter, not a rewrite.** That is the entire reason for the boundary. Don't build the second adapter now — just don't make it expensive to build later.
